available, the Drivers are then presented with their names and ratings, in descending order showing the top rated drivers first.

Support for PostgreSQL CTEs has been added to recent versions of Active Record in Ruby on Rails. Rails 7.1 adds a .with() method (See: *Rails 7.1 Construct Common Table Expressions*[16]) to generate a CTE statement. Prior to Rails 7.1 creating a CTE could be done using the activerecord-cte[17] gem.

To get some hands on experience with that, you'll adapt a simpler CTE to Active Record. From your terminal, run bin/rails console and use this Active Record code:

ruby/active_record_cte_with.rb

```
# use the ".." endless range
Trip.with(recently_rated:
          Trip.where.not(rating: nil).
          where(created_at: 30.days.ago..)
).
from("recently_rated").
count
```

Notice how this example combines multiple Active Record style queries into one. The .with() method is used as an outer query portion. The name recently_rated is given as a name to an inner query portion. Review the generated SQL. This query only counts rows, but consider using the techniques to create the earlier Driver SQL CTE as Active Record.

You've now seen how to perform Subqueries and CTEs in both SQL and Active Record.

As queries become more complex, it can be nice to encapsulate the details of complex queries and provide a simpler interface. Fortunately such a capability exists in PostgreSQL and it's called Database Views. Read on to see how those work and how to manage them with Ruby on Rails.

Introducing Database Views for Rideshare

In this section you'll work with *Database Views* which are a built-in object type for PostgreSQL.

Briefly, database views are a way to name and encapsulate a query and store that as an object within PostgreSQL.

16. https://blog.kiprosh.com/rails-7-1-construct-cte-using-with-query-method
17. https://github.com/vlado/activerecord-cte